

ALIKE: разбор кода

`alike.py` · `alnet.py` · `losses.py`

Архитектура, инференс и обучение
детектора ключевых точек и дескрипторов

Содержание

1. **Общая картина** — что такое ALIKE и как устроен пайплайн
2. `alnet.py` — backbone: энкодер + агрегация фич
3. `alike.py` — обёртка: детекция, мультимасштаб, инференс
4. `losses.py` — пять лоссов для детектора и дескриптора
5. **Поток данных** — от картинки до ключевых точек
6. **Глоссарий** — ключевые понятия

1. Общая картина

Ключевые идеи:

- Один энкодер → две плотные карты: `scores_map` + `descriptor_map`
- Субпиксельная детекция через **soft-argmax**
- Одна общая голова на детектор + дескриптор (эффективность)
- Обучение сквозным образом — детектор и дескриптор вместе

Иерархия классов:

```
LightningModule → BaseNet → ALNet → ALIKE
```

2. `alnet.py` — backbone

Роль: свёрточный энкодер + агрегация + головы.

5 классов:

Класс	Назначение
<code>BaseNet</code>	Абстрактная база (LightningModule)
<code>ConvBlock</code>	Два подряд Conv-BN-ReLU
<code>ResBlock</code>	Residual block со skip-connection
<code>PositionEncodingSine</code>	Опциональное синусоидальное PE
<code>ALNet</code>	Главный энкодер

2.1. ConvBlock и ResBlock

ConvBlock — простой двойной блок:

```
x → Conv3x3 → BN → ReLU → Conv3x3 → BN → ReLU
```

Используется только для **block1** (подъём 3→c1).

ResBlock — классический residual:

```
identity = x  
out = Conv → BN → ReLU → Conv → BN  
out = out + downsample(identity)  
out = ReLU(out)
```

downsample = 1×1 conv, если меняется число каналов.

stride=1 — даунсэмплинг делается отдельно через MaxPool.

2.2. PositionEncodingSine (опционально)

Добавляет **8 каналов** позиционной информации к RGB:

```
pe[0] = x_norm          pe[1] = y_norm
pe[2] = sin(x·2π)       pe[3] = cos(y·2π)
pe[4] = sin(x·8π)       pe[5] = cos(y·8π)
pe[6] = cos(x·32π)      pe[7] = cos(y·32π)
```

- Пересчитывается динамически под H×W
- Конкатенация по каналам: (B, 3, H, W) → (B, 11, H, W)
- В ALIKE по умолчанию pe=False

2.3. Архитектура ALNet

Энкодер (4 блока, даунсэмплинг $\times 32$):

Блок	Тип	Каналы	Разрешение
block1	ConvBlock	3(+8) $\rightarrow$ c1	H $\times$ W
block2	ResBlock	c1 $\rightarrow$ c2	H/2 $\times$ W/2
block3	ResBlock	c2 $\rightarrow$ c3	H/8 $\times$ W/8
block4	ResBlock	c3 $\rightarrow$ c4	H/32 $\times$ W/32

Агрегация фич — 3 режима:

- **sum** — 1 $\times$ 1 conv до **dim**, апсемпл, сложение
- **fpn** — top-down путь (Feature Pyramid)
- **cat** — 1 $\times$ 1 conv до **dim//4**, апсемпл, конкатенация (*по умолчанию*)

2.4. Головы (detector + descriptor)

Single head — одна общая голова:

```
self.convhead2 = resnet.conv1x1(dim, dim + 1)
```

Выход: **dim + 1** каналов

- **dim** каналов → дескрипторы
- **+1** канал → score ключевой точки

Разделение:

```
descriptor_map = x[:, :-1, :, :] # B×dim×H×W  
scores_map = torch.sigmoid(x[:, -1]).unsqueeze(1) # B×1×H×W
```

Score после **sigmoid** — это вероятность в $[0,1]$, не логит.

3. `alike.py` — обёртка модели

Наследуется от `ALNet`, добавляет:

- параметры детекции (`radius`, `top_k`, `scores_th`, `n_limit`)
- `SoftDetect` — превращает плотные карты в разреженные точки
- мультимасштабный инференс

Методы:

Метод	Назначение
<code>extract_dense_map</code>	forward + паддинг + L2-норм
<code>extract</code>	плотные карты → ключевые точки
<code>forward</code>	мультимасштабный инференс (numpy in)

3.1. extract_dense_map

Проблема: энкодер требует H, W кратны 32.

Решение: паддинг нулями до кратности 32, потом срез обратно.

```
h_ = math.ceil(h / 32) * 32
# ...pad...
scores_map, descriptor_map = super().forward(image)
# ...unpad...
descriptor_map = F.normalize(descriptor_map, p=2, dim=1)
```

Ключевой момент: дескрипторы **L2-нормализуются** по каналу.

→ косинусная метрика = евклидово расстояние.

3.2. `extract` — один проход

```
def extract(self, image):
    descriptor_map, scores_map = self.extract_dense_map(image)
    keypoints, descriptors, kptscores, scoredispersitys = \
        self.softdetect(scores_map, descriptor_map)
    return {'keypoints': ..., 'descriptors': ..., 'scores': ...,
            'score_dispersity': ..., 'descriptor_map': ..., 'scores_map': ...}
```

`SoftDetect` делает главную работу:

- субпиксельные координаты (soft-argmax)
- разреженные дескрипторы
- оценки + `score_dispersity`

3.3. forward — мультимасштаб

Вход: numpy H×W×3 (не тензор).

Алгоритм:

1. Ресайз, если слишком большое (`image_size_max`)
2. Конвертация в тензор (`ToTensor`)
3. **Пирамида масштабов** от 1.0 с шагом `scale_f = 2**0.5`
4. На каждом масштабе — `extract_dense_map` + `softdetect`
5. Сбор всех точек + сортировка по score

Денормализация координат:

```
keypoints = (keypoints + 1) / 2 * keypoints.new_tensor([[W_ - 1, H_ - 1]])
```

Из [-1,1] в пиксели **исходного** изображения.

3.4. forward — схема

```
np.array (H,W,3)
  | ToTensor + resize
  ▼
tensor (1,3,H,W)
  |
  ├── scale=1.0 → dense_map → softdetect → kp,desc,scores
  ├── scale=1/√2 → ...
  ├── scale=1/2 → ...
  └── ... до min_scale
  |
  ▼
cat всех точек → сортировка по scores → top-n_k
  |
  ▼
{'keypoints', 'descriptors', 'scores', ...}
```

Замечание: `score_dispersity` в `forward` **игнорируется** (нужен только при обучении).

4. `losses.py` — пять лоссов

Лосс	Что тренирует	Идея
<code>PeakLoss</code>	score-map	острые пики вместо плато
<code>ReprojectionLocLoss</code>	детектор	повторяемость локализации
<code>ScoreMapRepLoss</code>	детектор	score-map повторяема под warp
<code>DescReprojectionLoss</code>	дескрипторы	NLL по softmax (NRE)
<code>TripletLoss</code>	дескрипторы	hard negative mining
<code>local_similarity</code>	(опц.)	уникальность дескриптора

Все — `object` с `__call__`, а не `nn.Module`.

4.1. PeakLoss

Цель: не дать score-map стать uniform.

```
scores_kpts = pred['scores'][idx][:n_original]
valid = scores_kpts > self.scores_th
loss_peaky = pred['score_dispersity'][idx][valid]
loss_mean += loss_peaky.sum()
```

- Берём точки с уверенностью выше порога
- Минимизируем `score_dispersity` — «мягкую» меру плоскости карты
- Чем меньше — тем острее пики

Источник `score_dispersity` — `SoftDetect`.

4.2. ReprojectionLocLoss

Цель: ключевые точки должны быть **повторяемыми**.

```
dist = correspondences[idx]['dist']
scores0 = correspondences[idx]['scores0'].detach()[ids0_d]
valid = (scores0 > th) * (scores1 > th)
reprojection_errors = dist[ids0_d, ids1_d][valid]
loss = reprojection_errors.mean()
```

- `ids0_d`, `ids1_d` — **истинные** пары (через warp)
- `dist` — ошибка репроекции
- `.detach()` — scores только как фильтр
- Минимизируем среднюю ошибку

4.3. ScoreMapRepLoss

Цель: score-map должна «подтверждаться» на другой картинке.

```
# 1. Повторяемость score через grid_sample
scores_kpts01 = F.grid_sample(scores_map1, kpts01, ...)
s0 = scores_kpts01 * correspondences[idx]['scores0']

# 2. Repetability через similarity-map (detach!)
pmf01 = ((sim_map_01.detach() - 1) / T).exp()
repetability01 = torch.diag(F.grid_sample(pmf01, kpts01, ...))

# 3. Loss
loss01 = (1 - repetability01) * s0 * len(s0) / s0.sum()
```

Ключевое: `.detach()` на similarity-map → тренируем **только детектор**.

4.4. DescReprojectionLoss (NRE)

Цель: обучить дескрипторы так, чтобы правильный матч был **вероятностным максимумом**.

```
# Softmax по всем пикселям другой карты
pmf01 = softmax((sim_map_01 - 1) / T, dim=-1)

# Вероятность попасть в правильное место
C01 = diag(F.grid_sample(pmf01, kpts01, ...))

# Плюс бин "выброс" (outlier) — как в NRE
sim_out = cat([sim_out, ones], dim=1) # (N, h*w + 1)
C01_out = softmax(sim_out)[: , -1]

# Loss = NLL
loss = -log(cat([C01, C10, C01_out, C10_out])).mean()
```

Идея NRE: добавить uniform-бин «нигде». Точка, которая не матчится, уходит

В НЕГО

4.5. TripletLoss

Цель: правильный матч ближе к запросу, чем самый похожий негатив.

```
# Positive – через grid_sample
positive = diag(F.grid_sample(sim_map, kpts, ...))

# Hard negative mining
dist = correspondences[idx]['dist']
cosim = desc0 @ desc1.t()
cosim[dist < th] = -2 # safe radius 5 пикселей
negatives = cosim.sort(descending=True).values[:, 0]

# Loss
loss = relu(margin - positive + negatives).mean()
```

Safe radius — не штрафует за похожесть на соседей истинного матча.

4.6. `local_similarity` (вспомогательная)

Цель: оценить **уникальность** дескриптора.

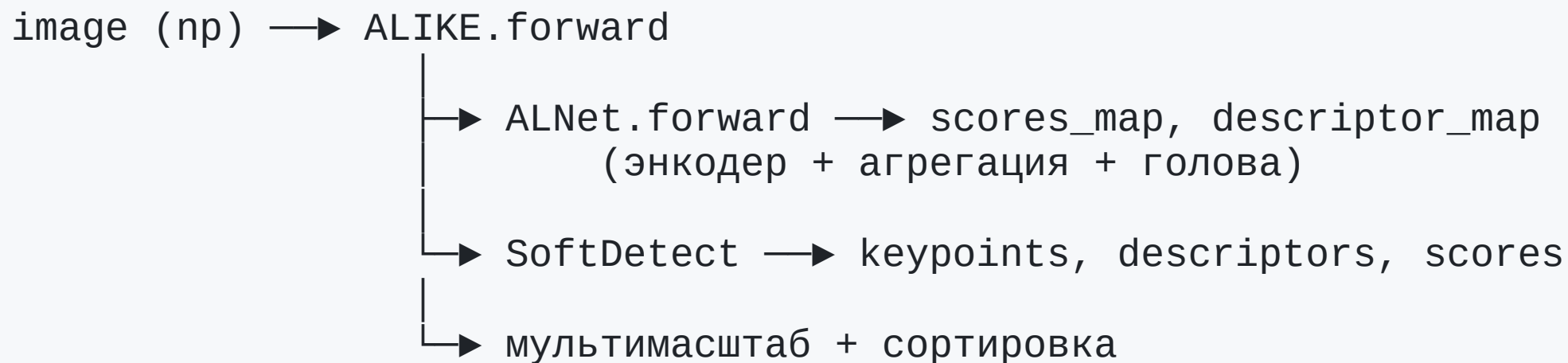
```
radius = 2
ksize = 5                                # 2*2+1
# Патч 5×5 вокруг точки
desc_patches = unfold(descriptor_map, ...)[kpts]
local_sim = einsum('nsd,nd->ns', desc_patches, descriptors) # (N, 25)
local_sim_sorted = sort(local_sim, descending=True).values
local_sim_mean = local_sim_sorted[:, 4:].mean(dim=1) # skip top-4
```

Почему skip top-4:

- дескриптор точки = билинейная интерполяция **до 4 пикселей**
- эти 4 всегда похожи на свой дескриптор → «загрязняют» метрику
- среднее по остальным 21 — честная мера уникальности

5. Поток данных

ИНФЕРЕНС



ОБУЧЕНИЕ

